

Linux Device Drivers- Syllabus

- Course duration – 60 hours (Total)

Linux - User space Refresh - 6 hours

Topic	Sub Topics	Hours
Linux Kernel compilation on X86 Desktop machine (OSL flow model)	- Linux OS (user space) vs Linux Kernel - Types of Devices in Linux OS - Kernel Source Tree with git and wget commands. - Explore Kernel source tree structure - Configuring, Building and Installing customized Kernel. - Boot process on X86 machine Lab Tasks 1. Download latest kernel version on Ubuntu machine. Configure debug kernel options. Boot Ubuntu machine with customized kernel. 2. Open grub.cfg file explore the options of new kernel configuration	4.5
Embedded Linux	- Overview of Embedded Linux System Architecture - Boot loader, root file system, Boot process on ARM - Cross compilation, Tool Chain installation. Lab Tasks 1. Flash Raspberry Pi SD image, 2. Cross compile and porting GPIO/I2C application on Raspberry Pi board 3. Build customized kernel for Raspberry Pi, port on	3
Modules & Device drivers	- Mechanism vs. Policy - How Applications Use Device Drivers - Walking Through a System Call Accessing a Device - Error Numbers - printk() - The module driver() Macros - Module parameters, Exporting Modules Lab Tasks 1. Set up the environment to compile kernel module on Raspberry PI board. 2. Write a kernel module with capabilities 3. Access kernel internals from your module.	2
Character Devices	- Device Nodes - Major and Minor Numbers - Reserving Major/Minor Numbers - Accessing the Device Node - Registering the Device - udev - dev printk() and Associates - file operations Structure - Driver Entry Points - The file and inode Structures - Miscellaneous Character Drivers	4.5

	Lab Tasks Implement character device driver on Raspberry Pi to read memory map	
Embedded Linux build system with Yocto	- Yocto poky reference build system. - Building a system image. - Writing a minimal recipe, Adding dependencies. - Development workflow with bitbake. - Adding the custom application. - Adding the custom library dependent application. - Adding custom kernel module. - Change the kernel version and apply kernel patches Lab Tasks 1. Load Kernel image/cpio/ext4 with QEMU platform 2. Build Raspberry pi Image with Yocto framework. Start from meta-raspbeerypi layer	4.5
Memory Management and Allocation	- Virtual and Physical Memory - Memory Zones - Page Tables - kmalloc() - get free pages() - vmalloc(), VM Split, VMA basics - Slabs and Cache Allocations - 64 bit Virtual address translation - NUMA nodes Lab Tasks 1. write kernel module to show information about kernel segments available on Raspberry.vmalloc region, low mem region	6
Memory mapped I/O and I/O mapped I/O	- Transferring Between Spaces - put(get) user() and copy to(from) user() - Direct Transfer: Kernel I/O and Memory Mapping - Mapping User Pages - Memory Mapping - User-Space Functions for mmap() - Driver Entry Point for mmap() - Accessing Files from the Kernel - Memory Barriers - Allocating and Mapping I/O Memory - Accessing I/O Memory Lab Tasks 1. Write kernel module to display memory map of raspberry	3
Interrupt handling	- What are Interrupts and Exceptions? - Exceptions - Asynchronous Interrupts - MSI - Enabling/Disabling Interrupts - What You Cannot Do at Interrupt Time - IRQ Data Structures - Installing an Interrupt Handler	4.5

	1. Top and Bottom Halves 2. Softirqs 3. Tasklets 4. Work Queues 5. New Work Queue API 6. Creating Kernel Threads 7. Threaded Interrupt Handlers 1.h Interrupt Handling in User-Space Lab Tasks 1. write kernel module to register shared interrupt with ex-isting interrupt, print messages from ISR and bottom half modules. 2. write kernel module to handle interrupt in user space by using Async I/O	
Unified Device Model and sysfs	- Unified Device Model - Basic Structures - Real Devices - sysfs - kset and kobject examples Lab Tasks Program to create sysfs file entries with store and show callbacks	1.5
Device Trees	- What are Device Trees? - What Device Trees Do and What They Do Not Do - Device Tree Syntax - Device Tree Walk Through - Device Tree Bindings - Device Tree support in Boot Loaders - Using Device Tree Data in Drivers - Coexistence and Conversion of Old Drivers Lab Tasks Explore device tree of raspberry PI, modify device tree and regenerate new dtb. Boot RP with modified dtb	1.5
Platform drivers	- What are Platform Drivers? - Main Data Structures - Registering Platform Devices - An Example - Hardcoded Platform Data - The New Way: Device Trees Lab Tasks Go through Xilinx Ethernet platform driver	2
Kernel Synchronization	- Critical section - Mutex lock - Semaphore - Spin lock - Kernel threads - Synchronization in kernel threads, wait events Lab Tasks Create 2 Kernel threads, producer and consumer threads. Synchronize two threds with available tools.	4.5
I2C and SPI client driver	-I2C subsystem	2

	-I2C Send/Receive data -SPI Subsystem Lab Tasks Develop I2C/SPI client Kernel modules for display device /BMP180 sensor	
PCI	- What is PCI? - PCI Device Drivers - Locating PCI Devices - Accessing Configuration Space - Accessing I/O and Memory Spaces PCI Express - PCI DMA - Allocate consistent DMA - Scatter and gather allocation - PCI interrupt handlers - PCI utilities Lab Tasks Go through PCI-Ethernet driver. Add debug statements Run network traffic, observe the transmit and receive path flow	4.5
USB drivers	- What is USB? - USB Topology - Terminology - Endpoints - Descriptors - USB Device Classes - USB Support in Linux - Registering USB Device Drivers - Moving Data Lab Tasks Go through USB skeleton device Go through USB enumeration process with USB analyzer logs Lib USB	2
Kernel Threads	Kernel threads & Synchronization	2
Monitoring and Debugging	- Debuginfo Packages - Tracing and Profiling - sysctl - SysRq Key - oops Messages debugging - Kernel Debuggers - debugfs Use perf, eBPF, addr2line, kprobe, Debug with performance utilities, Explore phoronix utility Kernel Core Dumps	